

Lab Program -02

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(X_train_full, y_train_full), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Full training set shape:", X_train_full.shape)
print("Test set shape:", X_test.shape)

# Normalize pixel values
X_train_full = X_train_full / 255.0
X_test = X_test / 255.0

# Split training into train and validation
X_train, X_val = X_train_full[:-5000], X_train_full[-5000:]
y_train, y_val = y_train_full[:-5000], y_train_full[-5000:]

# Add channel dimension (needed for Conv2D)
X_train = X_train[..., np.newaxis]
X_val = X_val[..., np.newaxis]
X_test = X_test[..., np.newaxis]

# Set seed for reproducibility
tf.random.set_seed(42)

# Define the CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, kernel_size=3, padding="same", activation="relu", kernel_initializer="he_normal",
input_shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(64, kernel_size=3, padding="same", activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Dense(128, activation="relu", kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation="softmax")
])

# Compile the model
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="nadam",
    metrics=["accuracy"]
)

# Train the model
model.fit(X_train, y_train, epochs=10, validation_data=(X_val, y_val))

# Evaluate the model
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test accuracy:", test_accuracy)
```

Lab Program -03

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.callbacks import EarlyStopping, ModelCheckpoint
from keras.optimizers import SGD, Adadelta, Adam, RMSprop, Adagrad, Nadam, Adamax

# Set seed for reproducibility
SEED = 2017

# Load data
data = pd.read_csv('Data/winequality-red.csv', sep=';')
y = data['quality']
X = data.drop(['quality'], axis=1)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, random_state=SEED)

# Create model function
def create_model(opt):
    model = Sequential()
    model.add(Dense(100, input_dim=X_train.shape[1], activation='relu'))
    model.add(Dense(50, activation='relu'))
    model.add(Dense(25, activation='relu'))
    model.add(Dense(10, activation='relu'))
    model.add(Dense(1, activation='linear')) # Regression output
    return model

# Create callbacks function
def create_callbacks(opt):
    callbacks = [
        EarlyStopping(monitor='val_accuracy', patience=200, verbose=2),
        ModelCheckpoint('checkpoints/optimizers_best_' + opt + '.h5',
                        monitor='val_accuracy', save_best_only=True, verbose=0)
    ]
    return callbacks

# Optimizers dictionary
opts = {
    'sgd': SGD(),
    'sgd-0001': SGD(learning_rate=0.0001, decay=0.00001),
    'adam': Adam(),
    'adadelta': Adadelta(),
    'rmsprop': RMSprop(),
    'rmsprop-0001': RMSprop(learning_rate=0.0001),
    'nadam': Nadam(),
    'adamax': Adamax()
}
```

```

batch_size = 128
n_epochs = 1000
results = []

# Loop through the optimizers
for opt in opts:
    model = create_model(opt)
    callbacks = create_callbacks(opt)
    model.compile(loss='mse', optimizer=opts[opt], metrics=['accuracy'])

    hist = model.fit(
        X_train.values, y_train,
        batch_size=batch_size,
        epochs=n_epochs,
        validation_data=(X_val.values, y_val),
        verbose=0,
        callbacks=callbacks
    )

    best_epoch = np.argmax(hist.history['val_accuracy'])
    best_acc = hist.history['val_accuracy'][best_epoch]

    best_model = create_model(opt)
    best_model.load_weights('checkpoints/optimizers_best_' + opt + '.h5')
    best_model.compile(loss='mse', optimizer=opts[opt], metrics=['accuracy'])

    score = best_model.evaluate(X_test.values, y_test, verbose=0)
    results.append([opt, best_epoch, best_acc, score[1]])

# Convert results to DataFrame
res = pd.DataFrame(results, columns=['optimizer', 'epochs', 'val_accuracy', 'test_accuracy'])

# Display results
print(res)

```

Lab Program -04

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
import os

# Define dataset directories
data_dir = "path/to/dataset" # Change this to your dataset path
train_dir = os.path.join(data_dir, "train")
val_dir = os.path.join(data_dir, "val")

# Define parameters
img_size = (224, 224)
batch_size = 32

```

```

num_classes = len(os.listdir(train_dir)) # Assuming each subdirectory is a class
epochs = 10 # Adjust as needed

# Data Augmentation and Preprocessing
datagen_train = ImageDataGenerator(
    rescale=1.0 / 255,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    validation_split=0.2 # Split train into train/val
)

datagen_val = ImageDataGenerator(rescale=1.0 / 255)

train_generator = datagen_train.flow_from_directory(
    train_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

val_generator = datagen_val.flow_from_directory(
    val_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

# Load Pretrained Model
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False # Freeze the base model

# Add Custom Layers
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
out = Dense(num_classes, activation='softmax')(x)

# Create Model
model = Model(inputs=base_model.input, outputs=out)

# Compile Model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train Model
model.fit(
    train_generator,
    validation_data=val_generator,

```

```

    epochs=epochs,
    steps_per_epoch=train_generator.samples // batch_size,
    validation_steps=val_generator.samples // batch_size
)

# Fine-tune the model by unfreezing some layers
base_model.trainable = True
for layer in base_model.layers[:100]: # Keep early layers frozen
    layer.trainable = False

# Re-compile with lower learning rate
model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Fine-tuning Training
model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=epochs // 2,
    steps_per_epoch=train_generator.samples // batch_size,
    validation_steps=val_generator.samples // batch_size
)

# Save Model
model.save("fine_tuned_resnet50.h5")

```

LAB Program -06

```

import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# Load MNIST data
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize and reshape
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# Add Gaussian noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# Build the autoencoder model

```

```

input_img = Input(shape=(28, 28, 1))

# Encoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2, 2), padding='same')(x)

# Decoder
x = Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2, 2))(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Compile model
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Train the model
autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)

# Predict on test data
decoded_imgs = autoencoder.predict(x_test_noisy)

# Visualize results
n = 15
plt.figure(figsize=(20, 4))
for i in range(n):
    # Noisy input
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    ax.axis('off')

    # Denoised output
    ax = plt.subplot(3, n, i + n + 1)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    ax.axis('off')

    # Ground truth
    ax = plt.subplot(3, n, i + 2 * n + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    ax.axis('off')

plt.tight_layout()
plt.show()

```

Program -07

```
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset from CSV
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv'
df = pd.read_csv(url, parse_dates=['Date'])

# Plot to visualize
plt.figure(figsize=(10, 4))
plt.plot(df["Temp"])
plt.title("Daily Min Temperatures")
plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.show()

# Convert to float32
temps = df["Temp"].values.astype(np.float32)

# Normalize for better training
mean = temps.mean()
std = temps.std()
temps = (temps - mean) / std

# Function to create sequences
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i + window_size])
        y.append(data[i + window_size])
    return np.array(X), np.array(y)

# Create input/output sequences
window_size = 7 # Predict next day from previous 7 days
X, y = create_sequences(temps, window_size)

# Reshape for RNN input: (samples, time steps, features)
X = X.reshape((X.shape[0], X.shape[1], 1))

# Split into train/test sets
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Build the model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(64, return_sequences=True, activation='tanh', input_shape=(window_size, 1)),
    SimpleRNN(32, activation='tanh'),
    Dense(1)
```

```

])

# Compile the model
model.compile(optimizer='adam', loss='mse')
model.summary()

# Early stopping
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Train the model
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=100,
    batch_size=32,
    callbacks=[early_stop]
)

# Predict on test data
y_pred = model.predict(X_test)

# Denormalize predictions and actual values
y_pred_denorm = y_pred * std + mean
y_test_denorm = y_test * std + mean

# Plot actual vs predicted
plt.figure(figsize=(10, 4))
plt.plot(y_test_denorm, label='Actual')
plt.plot(y_pred_denorm, label='Predicted')
plt.legend()
plt.title("Actual vs Predicted Temperatures")
plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.show()

```